Research setup guide

FleetCentric.ai research component · first-time setup for team members · November 2026

In short: you install WSL (a small Linux that runs inside Windows) once, add one setting, then double-click **Research Hub.bat**. From then on everything is buttons in a web page. Plan for 30 to 45 minutes the first time; most of it is waiting for downloads.

What the research component is

The **research/** folder tests whether shuttle routes can be planned automatically, and re-planned when a road is blocked, on a free cloud server (512 MB memory, a tenth of one CPU). It is separate from the website: nothing in it changes the website, the backend or the database. For the full story, results and next steps, read the team overview document *FleetCentric Research: Overview for the Team*.

What you need

Item	Why it is needed / how to get it
Windows 10 (2004 or newer) or Windows 11	WSL 2 needs a recent Windows.
Administrator rights	Only to turn on WSL in step 2.
Virtualization turned on	WSL runs a tiny virtual machine. Check Task Manager → Performance → CPU: "Virtualization: Enabled". It is on for most laptops.
Git	To get the code. git-scm.com/download/win
Python 3.10 or newer, on Windows	Runs the hub. python.org/downloads; tick **Add python.exe to PATH** in the installer.
About 2 GB free disk and internet	Ubuntu is about 1.5 GB; the Python packages about 300 MB. Internet is only needed for the first install.

Not needed: Docker, OSRM, Node, or the website's backend and database. The road travel times are already saved in the repo.

What the hub does, and what changed

The hub is a small web page that runs only on your own PC (at `http://127.0.0.1:8765`). It puts the whole research component in one place: you run experiments with buttons and see the results as charts, maps and tables. Under the hood it runs the same scripts as before, so the results are the same.

Task	Before the hub	With the hub
Set up Python	Type venv and pip commands in Ubuntu	Press **Install**
Check the limits work	Run `run_budget.py --check` and read kernel numbers	Press **Check setup**; four green ticks
Run an experiment	Type a long command in WSL and keep that window open	Press **Start now**, or **Schedule…** for tonight
Continue after a stop	Remember and retype the command	Press **Resume now**
See results	Open separate JSON and HTML files in results/	Tabs with charts and maps, refreshed every few seconds

Task	Before the hub	With the hub
Share results	Summarize, git add, commit and push by hand; others need the repo to look	Press **Publish** (or let it publish when a run ends); a shared web page updates for everyone
Keep the PC awake	Change Windows power settings	Automatic while a job runs or waits
Figures for the report	Screenshot or redraw them	PNG, CSV and LaTeX export buttons on each chart and table

Teammates who only want to look need none of this setup: the shared page is at **hassanh5n.github.io/FAST-Transport**.

Setup, once per PC

1 Get the code

Already have the repo? Open a terminal in its folder and run `git pull`. **New to it?** Clone it:

```
git clone https://github.com/hassanh5n/FAST-Transport.git
```

Why: the research/ folder comes with the repo. Keep the repo on a normal drive such as C: or D:.

2 Turn on WSL and install Ubuntu

Open PowerShell **as administrator** (Start, type PowerShell, right-click, Run as administrator):

```
wsl --install -d Ubuntu
```

Restart when asked. After the restart an Ubuntu window opens and asks for a new username and password. Pick anything and remember the password: you need it for `sudo`. While you type a password, nothing shows on screen. That is normal.

Why: the optimizer and the free-tier limits run in Linux. WSL is Linux inside Windows, so no second PC or Docker.

3 Install Python tools inside Ubuntu

In the Ubuntu window (Start → Ubuntu), run this and type your Ubuntu password:

```
sudo apt update && sudo apt install -y python3-venv python3-pip
```

Why: Ubuntu comes with Python but without the part that makes a private Python environment. The hub needs it to install OR-Tools, the route solver.

4 Turn on systemd (this applies the free-tier limits)

Still in Ubuntu, run:

```
printf '[boot]\nsystemd=true\n' | sudo tee -a /etc/wsl.conf
```

Then, in a normal PowerShell window, restart WSL so the setting takes effect:

```
wsl --shutdown
```

Why: systemd is what caps each run at 512 MB of memory and 10% of one CPU, the same as a free cloud server. Without it, the "limited" results would mean nothing.

5 Open the hub

In File Explorer, open the repo's **research** folder and double-click **Research Hub.bat**. A black window opens and your browser shows the hub.

Why: the black window is the hub itself. Leave it open while you use the hub; closing it closes the hub.

6 Check setup and install

Open the **Run & publish** tab and press **Check setup** (10 to 20 seconds). If "Python environment" shows a cross, press **Install it** and wait 2 to 5 minutes. Then press **Check setup** again.

> **Done when all four are green:** WSL starts · Python environment · OR-Tools installed · Free-tier limits apply. The last check really starts a limited run and reads the limits back, so four green ticks means you are ready.

Using it day to day

- **Pull first.** Run `git pull` before you start, so you have everyone's latest results.
- **Tell the team before a long run.** Two people running the same experiment would both write to the same results file and clash in git.
- **Start, Resume, Stop.** On Run & publish. Stopping keeps every finished run; Resume continues from there.
- **Schedule.** Press **Schedule…**, pick a time such as 01:00, keep the hub window open and the laptop plugged in. The hub stops Windows from sleeping until the job ends.
- **Publish.** Tick "Publish to the shared page when a run finishes", or press **Publish results**. It commits and pushes only the research/ folder; the shared page updates a minute or two later. You need push access to the GitHub repo; ask the repo owner to add you. Without it you can still run and view locally.
- **Tabs:** Overview (questions and progress), Experiments (charts), Runs (each run on a map), Data (stops and routes), Documents (logbook, README, papers), Run & publish (controls).

If something goes wrong

Problem	Fix
Black window says Python is not installed	Install Python from python.org with **Add python.exe to PATH** ticked, then double-click the .bat again.
WSL error `HCS_E_HYPERV_NOT_INSTALLED` or "virtualization is not enabled"	In an administrator PowerShell run `wsl --install --no-distribution` and `bcdedit /set hypervisorlaunchtype auto`, then restart. If it still fails, turn on Intel VT-x or AMD SVM in the BIOS.
Install fails with "ensurepip is not available"	Step 3 was skipped. Run its apt command, then press Install again.
"Free-tier limits apply" shows a cross	systemd is off. Redo step 4, run `wsl --shutdown`, then Check setup. If `/etc/wsl.conf` already had a [boot] section, fix it with `sudo nano /etc/wsl.conf`.
"No free port between 8765 and 8780"	A hub is already open. Use that browser tab, or close the old black window first.
Page looks old or broken	Press Ctrl+F5 to reload it.
Publish says the push was rejected	Someone pushed first. Run `git pull`, then Publish again.
The PC slept and a run stopped	Press **Resume now**. Only the run in progress is lost.

Only if you need to rebuild the road data

Docker and OSRM are needed only to rebuild the travel-time matrices (if the stops change) or for the road-block test planned next. The steps are in `research/README.md`, section "Build the Karachi road network". Everything else works without them.